See how we convert the basic generator into a lazy enumerator with the call to lazy after the end of the block.

Calling the first method on this with the argument 10 returns the numbers 1 through 10, but this doesn't exercise the method's lazy characteristics. Lets instead get the first 10 multiples of three.

```
p InfiniteStream.new.all
    .select { (_1 % 3).zero? }
    .first(10)
```

produces:

```
[0, 3, 6, 9, 12, 15, 18, 21, 24, 27]
```

Without the lazy enumerator, the call to select would effectively never return, as select would try to read all the values from the generator. But the lazy version of select only consumes values on demand, and in this case the subsequent call to first only asks for 10 values.

Let's make this a little more complex—how about multiples of 3 whose string representations are palindromes?

```
def palindrome?(n)
  n = n.to_s
  n == n.reverse
end

p InfiniteStream.new.all
    .select { (_1 % 3).zero? }
    .select { palindrome?(_1) }
    .first(10)
```

produces:

```
[0, 3, 6, 9, 33, 66, 99, 111, 141, 171]
```

Remember that our lazy filter methods simply return new Enumerator objects? That means we can split up the previous code:

```
multiple_of_three = InfiniteStream.new.all.select { (_1 % 3).zero? }

p multiple_of_three.first(10)

m3_palindrome = multiple_of_three.select { palindrome?(_1) }

p m3_palindrome.first(10)
```

produces:

```
[0, 3, 6, 9, 12, 15, 18, 21, 24, 27]
[0, 3, 6, 9, 33, 66, 99, 111, 141, 171]
```

You could also code up the various predicates as free-standing procs, if you feel it aids readability or reusablility.

```
multiple_of_three = -> n { (n % 3).zero? }
palindrome = -> n { n = n.to_s; n == n.reverse }

p InfiniteStream.new
    .all
    .select(&multiple_of_three)
    .select(&palindrome)
    .first(10)
```